

Statistical and Machine Learning

Double descent

Additional Exercise: Double descent

An illustrative example taken directly from the R demo at **this website**. The blog post was inspired by **this video**.

Why does it happen?

The key lies in the relationship between the number of data points (n) and the number of features (k).

When a model has fewer parameters than data points ($k < n$), no single model fits the training data perfectly – the algorithm must average or compromise, which introduces bias. As k grows toward n , the model gains just enough flexibility to fit the data exactly, but with almost no room to average anything out. This is the worst of both worlds: high variance, poor generalization, and the peak of the test error curve.

Once k exceeds n , something shifts. Now there are many models that fit the training data perfectly, and the learning algorithm effectively averages over them. This averaging reduces variance. The more overparameterized the model becomes – the more k dominates n – the greater the number of degrees of freedom available to exploit, and the better the generalization can become.

More formally, the degrees of freedom in a problem scale roughly as $\max(n, k)/\min(n, k)$. In the overparameterized regime, this grows with k , providing more and more opportunity to smooth out noise. That's why making a neural network larger, past the interpolation threshold, can actually improve test performance rather than hurt it.

Generate a univariate data set

```
make_data <- function(n, sigma){
  x <- 2*(runif(n) - 0.5)
  y <- x * (x-0.6) * (x+0.6) + sigma * rnorm(n)
  data.frame(x=x, y=y)
}
```

Add new features to the data set

```
model_matrix <- function(dat, cutoffs, s_noise=0){
  n <- nrow(dat)
  N <- length(cutoffs)
  extra_features <- outer(dat$x, cutoffs, ">") + s_noise*rnorm(N*n)
  cbind(1, dat$x, extra_features)
}
```

Fitting the model

To fit the model, add K of these features at random. Use the Moore-Penrose pseudoinverse (the `ginv` function from the MASS package) of the model matrix to get a vector of regression coefficients `bb`

```
library(MASS)

fit_model <- function(dat, K, s_noise=0.01){
  # N is the number of random features added
  # dat is a univariate data set with columns x, y
  a <- min(dat$x)
  b <- max(dat$x)

  cutoffs <- runif(K)*(b-a) + a # random cutoffs

  bb <- ginv(model_matrix(dat, cutoffs, s_noise)) %*% dat$y
  list(bb=bb, cutoffs=cutoffs)
}

predict_model <- function(model, dat){
  # predict from the model from data set with column x
  model_matrix(dat, model$cutoffs, 0) %*% model$bb
}

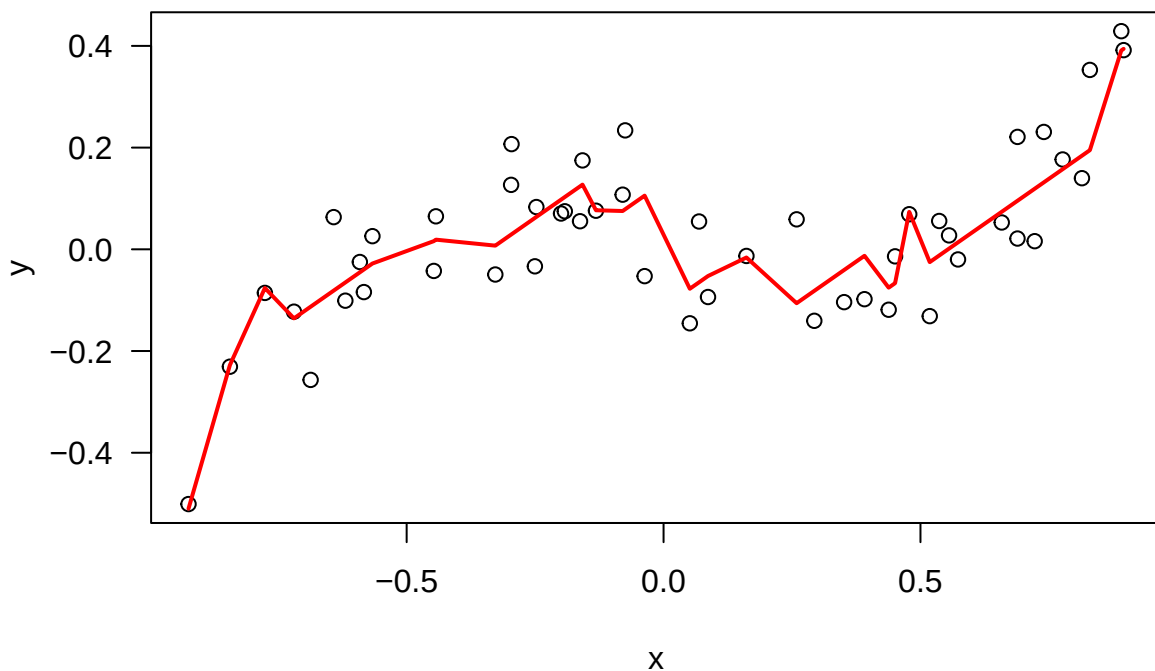
```

Good fit it to the training data

```
set.seed(2510)
dat <- make_data(50, 0.1)
model <- fit_model(dat, 20)
plot(dat, main="Training data and fitted model", las=1)
dat$pred <- predict_model(model, dat)
lines(dat$x[order(dat$x)], dat$pred[order(dat$x)], "l", col="red",
      lwd=2)

```

Training data and fitted model



Calculate variance and MSE

```
calc_test_error <- function(model, nrow_test, sigma_test, n_reps){

  mse <- rep(0, n_reps)
  for (i in 1:n_reps){
    dat_test <- make_data(nrow_test, sigma_test)
    pred <- predict_model(model, dat_test)
    mse[i] <- mean((pred - dat_test$y)^2)
  }
  mean(mse)
}

calc_variance <- function(n_pts, n_model, sigma, n_reps){

  x_test <- seq(-1, 1, len=n_pts)
  fitted <- matrix(0, nr=n_reps, nc=length(x_test))

  for (i in 1:n_reps){
    dat <- make_data(n_pts, sigma)
    model <- fit_model(dat, n_model)
    fitted[i,] <- predict_model(model, data.frame(x=x_test))
  }
  mean(apply(fitted, 2, var))
}
```

Double descent

```
set.seed(2510)
K <- 50
model_reps <- 10
dat <- make_data(10, 0.1)
err <- vars <- matrix(0, nr=K+1, nc=model_reps)
for (i in 0:K){
  print(i)
  for (j in 1:model_reps){
    model <- fit_model(dat, i)
    err[i+1, j] <- calc_test_error(model, 100, 0.1, 1000)
    vars[i+1, j] <- calc_variance(10, i, 0.1, 1000)
  }
}
```

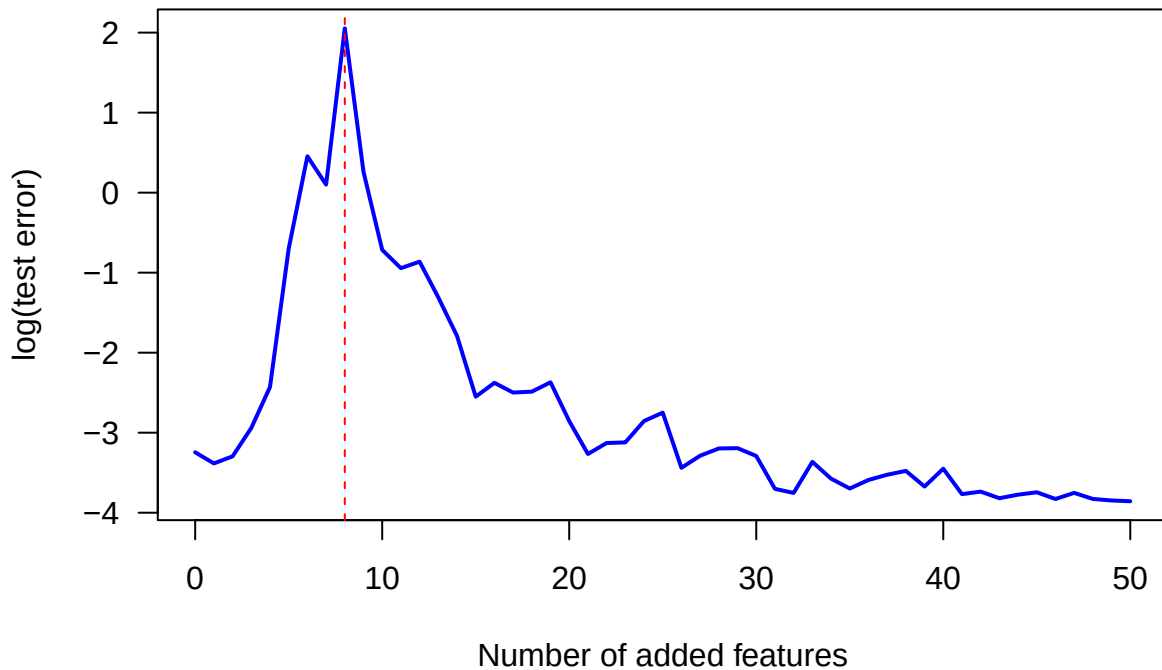
```
## [1] 0
## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] 5
## [1] 6
## [1] 7
## [1] 8
## [1] 9
## [1] 10
## [1] 11
```

```
## [1] 12
## [1] 13
## [1] 14
## [1] 15
## [1] 16
## [1] 17
## [1] 18
## [1] 19
## [1] 20
## [1] 21
## [1] 22
## [1] 23
## [1] 24
## [1] 25
## [1] 26
## [1] 27
## [1] 28
## [1] 29
## [1] 30
## [1] 31
## [1] 32
## [1] 33
## [1] 34
## [1] 35
## [1] 36
## [1] 37
## [1] 38
## [1] 39
## [1] 40
## [1] 41
## [1] 42
## [1] 43
## [1] 44
## [1] 45
## [1] 46
## [1] 47
## [1] 48
## [1] 49
## [1] 50
```

```
plot_col <- rep("blue", K+1)
plot_col[1:9] <- "forestgreen" # used later
df <- data.frame(vars=rowMeans(log(vars)),
                  err=rowMeans(log(err)),
                  plot_col=plot_col)

plot(0:K, df$err, "l", xlab="Number of added features",
     ylab="log(test error)", las=1, main="Double Descent",
     col="blue", lwd=2)
abline(v=8, lty=2, col="red")
```

Double Descent



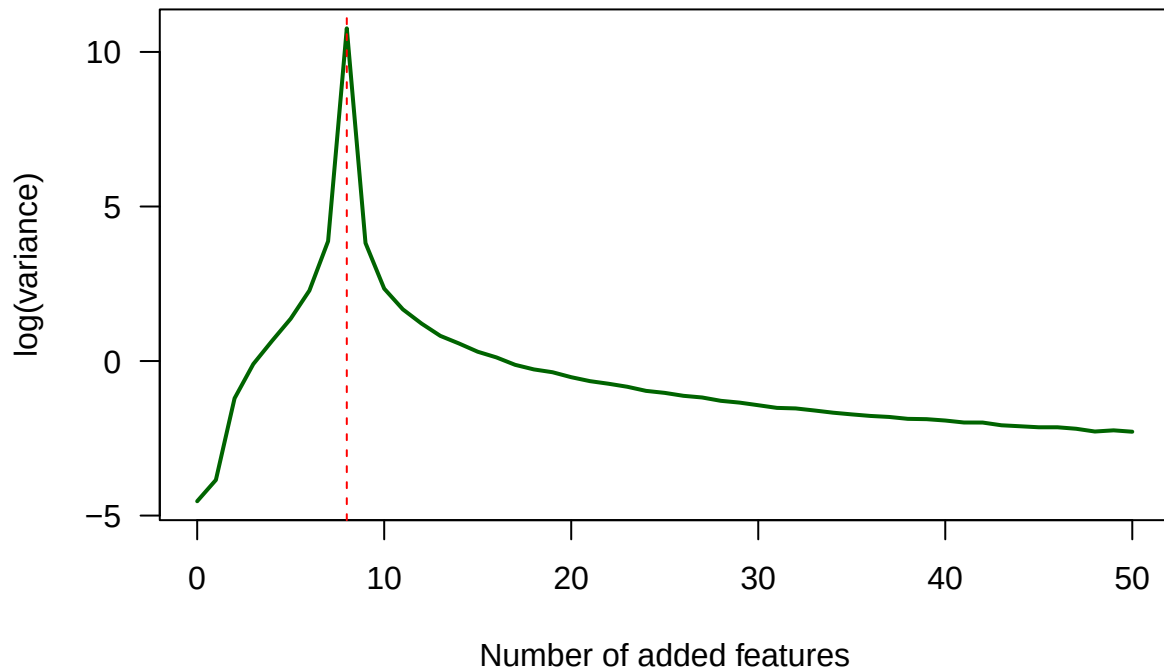
The red dotted line shows that the maximum test error occurs at $K = 8$ added features. Why 8? This is because our model matrix has $K + 2$ columns and we have 10 data points. So when $K = 8$ we get a square model matrix, which means that there is exactly one possible model which can output the training y values given the training x values. This is the point where the maximum amount of overfitting occurs.

Note also that the models with a very large number of added features had the lowest test error; even lower than the models with just a few added features. Just like in real deep learning!

Bias variance trade-off

```
plot(0:K, df$vars, "l", xlab="Number of added features",  
     ylab="log(variance)", las=1, main="Model variance",  
     lwd=2, col="darkgreen")  
abline(v=8, lty=2, col="red")
```

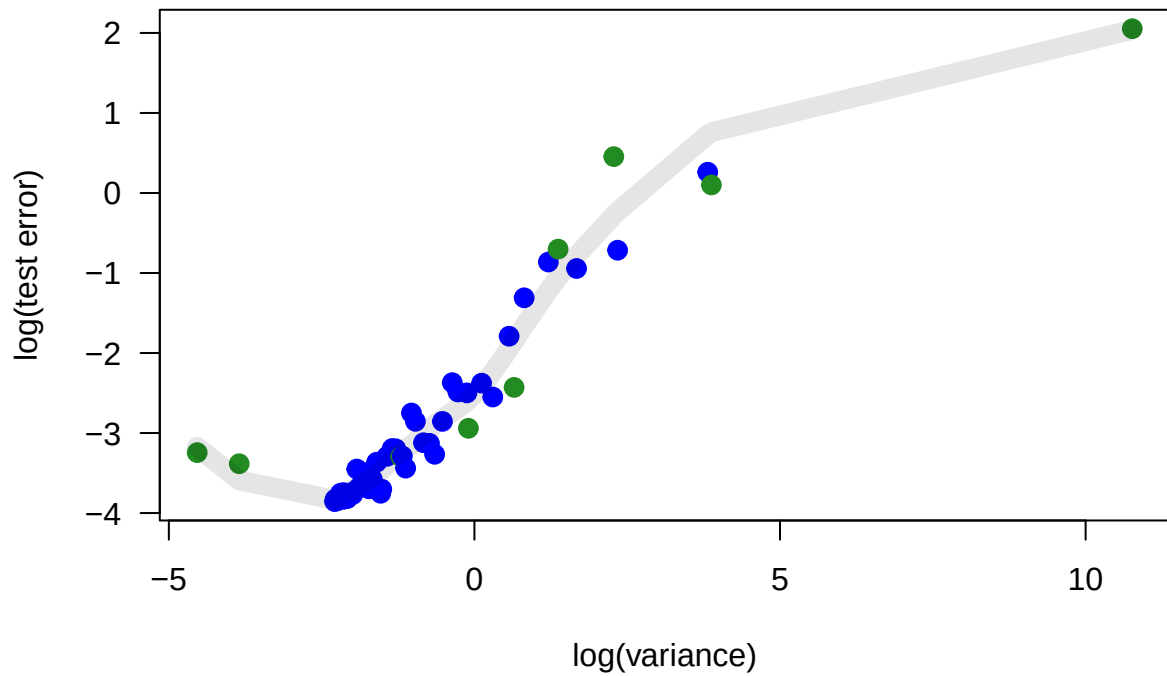
Model variance



This is the key to understanding why double descent doesn't contradict the traditional notion of the bias-variance tradeoff. When we plot the U-shaped curve in the bias-variance tradeoff, it's the variance of the model which we should be plotting on the x -axis, not the "model complexity" or "model size" or some other notion of that sort.

```
df$K <- 0:K
df <- df[order(df$vars), ]
plot(df$vars, df$err, col=df$plot_col,
     pch=19, xlab="log(variance)", ylab="log(test error)",
     las=1, cex=1.3)

loess_fit <- loess(df$err ~ df$vars, span = 0.6)
lines(df$vars, predict(loess_fit), col=rgb(0, 0, 0, 0.1), lwd=10)
```



When we plot the test error against the variance, we do get the familiar U-shaped curve after all. The green points are the models which are below the $K = 8$ threshold. The blue points are the models which are above the threshold. Together they form a nice familiar pattern.